COMPSCI 689
Lecture 20: Non-LinearFactor Analysis and Variational Autoencoders

Benjamin M. Marlin

College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

Factor Analysis: Probabilistic Model

- The probabilistic model/generative process for factor analysis is shown below:

$$p(\mathbf{X} = \mathbf{x}, \mathbf{Z} = \mathbf{z}) = p(\mathbf{X} = \mathbf{x}|\mathbf{Z} = \mathbf{z})p(\mathbf{Z} = \mathbf{z})$$
$$p(\mathbf{Z} = \mathbf{z}) = \mathcal{N}(\mathbf{z}; 0, I)$$
$$p(\mathbf{X} = \mathbf{x}|\mathbf{Z} = \mathbf{z}) = \mathcal{N}(\mathbf{x}; \mathbf{W}\mathbf{z} + \mu, \Psi)$$

- Ψ is restricted to be a positive, diagonal matrix. We can learn μ, or simply remove the data set mean and require $\mu = 0$.
- This model is commonly used for dimensionality reduction and generation, but learns a fundamentally linear relationship between $\mathbf{Z}$ and $\mathbf{X}$.

Autoencoders vs Factor Analysis

- Autoencoders and classical factor analysis can be applied to many of the same tasks including denoising and representation learning.
- Deep autoencoders can learn to represent data from complex manifolds using highly non-linear representations; however, they are not probabilistic models.
- Classical factor analysis is a probabilistic model, but the mean relationship between the latent variables and the visible variables is linear.
- What if we could merge the strengths of these two model classes and build deep, non-linear probabilistic autoencoders?

Non-Linear Factor Analysis

- This model generalizes the basic factor analysis model where the relationship between $\mathbf{z}$ and the mean of $\mathbf{X}$ is linear, to the case where the mean of $\mathbf{X}$ is generated from $\mathbf{z}$ using a deterministic non-linear transformation $f_{\mathbf{w}}(\mathbf{z})$ specified by a neural network.
- As with the basic factor analysis model $p(\mathbf{Z} = \mathbf{z}) = \mathcal{N}(\mathbf{z}; 0, I)$.
- The conditional probability of $\mathbf{X}$ is given by:

$$p(\mathbf{X} = \mathbf{x}|\mathbf{z}, \theta) = \mathcal{N}(\mathbf{x}; f_{\mathbf{w}}(\mathbf{z}), \Psi)$$

- $f_{\mathbf{w}}(\mathbf{z})$ represents a decoder/generator network. The model parameters are $\mathbf{w}$ and Ψ. Ψ is again a diagonal covariance matrix.

Non-Linear Factor Analysis: Joint Density

- The joint density of $\mathbf{X}$ and $\mathbf{Z}$ is thus given by:

$$p(\mathbf{X} = \mathbf{x}, \mathbf{Z} = \mathbf{z}|\theta) = \mathcal{N}(\mathbf{x}; f_{\mathbf{w}}(\mathbf{z}), \Psi)\mathcal{N}(\mathbf{z}; 0, I)$$

- The joint density values are always computable and we can sample efficiently from a learned model.
- However, the joint *distribution* of $\mathbf{X}$ and $\mathbf{Z}$ is no longer multivariate Gaussian.
- Further, the marginal probability of $\mathbf{X}$ has no closed form expression, and we can not learn the model using direct NLML minimization:

$$nlml(\mathcal{D}, \theta) = -\sum_{n=1}^{N} \log \left(\int p(\mathbf{X} = \mathbf{x}_n, \mathbf{Z} = \mathbf{z}|\theta) d\mathbf{z} \right)$$

Approximating the NLML

- In this section, we'll look at an approach to dealing with intractable NLML functions.
- This approach constructs an adjustable upper bound on the NLML function using a set of auxiliary distributions $q(\mathbf{Z} = \mathbf{z}|\phi_n) \approx p(\mathbf{Z} = \mathbf{z}|\mathbf{X} = \mathbf{x}, \theta)$, one per data case.
- The upper bound is constructed using a useful result called *Jensen's Inequality*.

Jensen's Inequality: Continuous Case

- Let $g(x)$ be an arbitrary real-valued function of $x \in \mathcal{X}$ and $q(X = x)$ be any probability density function over the same space, then the following inequality holds:

$$-\log\left(\int q(X=x)g(x)dx\right) \leq -\int q(X=x)\log(g(x))dx$$

$$-\log(\mathbb{E}_{q(X)}[g(x)]) \leq -\mathbb{E}_{q(X)}[\log(g(x))]$$

An Upper Bound on the NLML

$$
\begin{aligned}
nlml(\mathcal{D},\theta) &= -\sum_{n=1}^{N} \log\left(\int p(\mathbf{x}_n,\mathbf{z}|\theta)d\mathbf{z}\right) \\
&= -\sum_{n=1}^{N} \log\left(\int \frac{q(\mathbf{z}|\phi_n)}{q(\mathbf{z}|\phi_n)} p(\mathbf{x}_n,\mathbf{z}|\theta)d\mathbf{z}\right) \\
&= -\sum_{n=1}^{N} \log\left(\int q(\mathbf{z}|\phi_n)\frac{p(\mathbf{x}_n,\mathbf{z}|\theta)}{q(\mathbf{z}|\phi_n)}d\mathbf{z}\right) \\
&\leq -\sum_{n=1}^{N} E_{q(\mathbf{z}|\phi_n)}\left[\log\left(\frac{p(\mathbf{x}_n,\mathbf{z}|\theta)}{q(\mathbf{z}|\phi_n)}\right)\right] \\
&= -\sum_{n=1}^{N} E_{q(\mathbf{z}|\phi_n)}\left[\Big(\log p(\mathbf{x}_n,\mathbf{z}|\theta) - \log q(\mathbf{z}|\phi_n)\Big)\right] \qquad = Q(\mathcal{D},\theta,\phi)
\end{aligned}
$$

Minimizing the Q Function

- Our task is now to minimize the upper bound $Q(\mathcal{D}, \theta, \phi)$.
- If the distributions are such that the expectation in $Q(\mathcal{D}, \theta, \phi)$ can be computed analytically, we obtain an algorithm referred to as the *variational expectation maximization* or variational EM algorithm by alternately optimizing the auxiliary and primary parameters.

$$\text{E-Step: } \phi^{t+1} \leftarrow \arg\min_{\phi} Q(\mathcal{D}, \theta^t, \phi)$$

$$\text{M-Step: } \theta^{t+1} \leftarrow \arg\min_{\theta} Q(\mathcal{D}, \theta, \phi^{t+1})$$

- If the expectations in $Q(\mathcal{D}, \theta, \phi)$ can be computed analytically, we can also just jointly minimize $Q(\mathcal{D}, \theta, \phi)$ with respect to all parameters.

Stochastic Variational EM

- Even when using a simplified approximating distribution $q_n(\mathbf{z}|\phi_n)$, its still possible for the term $\mathbb{E}_{q(\mathbf{z}|\phi_n)}[\log p(\mathbf{x}_n, \mathbf{z}, |\theta)]$ to contain a computationally intractable integral.
- In such cases, we can consider a stochastic approximation to the upper bound based on drawing samples $z_{sn} \sim q_n(\mathbf{z}|\phi_n)$.
- We obtain an approximate objective:

$$\frac{1}{S}\sum_{s=1}^{S}\sum_{n=1}^{N}(\log p(\mathbf{x}_n, \mathbf{z}_{sn}, |\theta) - \log q(\mathbf{z}_{sn}|\phi_n))$$

Reparameterization Trick

- Note that while this approximation is a standard Monte Carlo approximation to an integral, it can not be directly used for learning the variational parameters. Why?
- The distribution we sampled from depends on the variational parameters, and we can't take gradients through sampling.
- Instead, we can often draw samples from a parameter-free base distribution, and then transform them into samples from $q(\mathbf{z}|\phi_n)$ using a deterministic, differentiable function.

Example: Diagonal Gaussian Re-parameterization

- Suppose that $q(\mathbf{z}|\phi)$ is a product of Gaussian marginals with mean m_k and standard deviation s_k on dimension k.
- We can sample $\mathbf{z}$ by sampling each of its dimensions z_k from the marginal $\mathcal{N}(m_k, s_k^2)$.
- To begin, sample $\eta_k \sim \mathcal{N}(0, 1)$ for each k.
- Now define $z_k = m_k + \eta_k s_k$ for each k.
- Then z_k is a sample from $\mathcal{N}(m_k, s_k^2)$ and $\mathbf{z}$ is a sample from $q(\mathbf{z}|\phi)$

Re-parameterization Trick

- This re-parameterization gives the approximate objective:

$$\frac{1}{S}\sum_{s=1}^{S}\sum_{n=1}^{N}(\log P(\mathbf{x}_n, \mathbf{z}_{ns}, |\theta) - \log q(\mathbf{z}_{ns}|\phi_n))$$

$$\mathbf{z}_{kns} = m_{kn} + \eta_{kns} s_{kn}$$

$$\eta_{kns} \sim \mathcal{N}(0, 1)$$

Learning with Stochastic Approximation

- On each learning iteration, given a set of samples η_{kns}, we can compute the gradient of the approximate objective and do gradient-based learning.
- Note that since all of the integrals are gone, we can implement the computation for $p(\mathbf{x}_n, \mathbf{z}_{ns}, |\theta)$ and $q(\mathbf{z}_{ns}|\phi_n)$ and use automatic differentiation to obtain the required gradients.
- This allows us to more easily build complex distributions for both p and q. So long as we can sample from q using re-parameterization, and both p and q are differentiable, we can apply this framework.

Amortized Variational Learning

- In the algorithm presented so far, we have an extra set of parameters ϕ_n for each data case n. Representing and optimizing these parameters can be quite costly.
- We can instead learn an auxiliary neural network model (called an inference network or a recognition network) that predicts the optimal parameters ϕ_n for the model $q(\mathbf{z}|\phi_n)$ using $\mathbf{x}_n$ as input.
- For a Gaussian mean-field approximating distribution, we construct a network that predicts the mean and standard deviation parameters: $m_\phi(\mathbf{x}_n)$ and $s_\phi(\mathbf{x}_n)$.
- These parameter prediction functions are usually different *output heads* of a common underlying network structure with its own set of learnable parameters ϕ.
- This is referred to as *amortized variational inference*.

Variational Autoencoder

- A variational autoencoder combines all of the ideas introduced in this lecture.
- The model is non-linear factor analysis. The learning approach minimizes the stochastic variational upper bound on the NLML with amortized variational mean field inference.
- For this specific model, we assume a factorized auxiliary distribution

$$q(\mathbf{z}|\mathbf{x}_n, \phi) = \prod_{k=1}^{K} q(z_k|\mathbf{x}_n, \phi) = \prod_{k=1}^{K} \mathcal{N}(z_k; m_\phi(\mathbf{x}_n)[k], s^2_\phi(\mathbf{x}_n)[k])$$

- $m_\phi(\mathbf{x}_n)[k]$ is the k^{th} dimension of the predicted mean vector and $s_\phi(\mathbf{x}_n)[k]$ is the k^{th} dimension of the predicted vector of standard deviations.

Variational Autoencoders

- This model family has the advantage that it is a full probabilistic model for $\mathbf{X}$.
- The neural network model used in the generator $\mathcal{N}(\mathbf{x}; f_{\mathbf{W}}(\mathbf{z}), \Psi)$ can model complex non-linear manifolds.
- The neural network models used in the recognition network enables optimization-free approximate inference during and after model learning.

Stochastic Variational Autoencoder